

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : ____ / ____ / ____

Mohsin Ibne Hossain

DATA BASE
Final Term!

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Table of Contents

SL.NO	Topic	Pages
1	Introduction to Sub-Query's	03-04
2	Single Row Sub-Query's	05-09
3	Multiple Row Sub-Query's	10-15
4	Introduction to Joining of Tables	16-17
5	Equi-Join	18
6	Non Equi-Join	19
7	Outer-Join	20-22
8	Self-Join	22
9	Exercise of Joining Query's	23-29
10	Views	30-39
11	Sequence	40-44
12	Relational Algebra	45-51
13	Normalization	52-63

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

#Subquery:-

⇒ A subquery is a query that appears inside another query statement.

Query Syntax:-

SELECT Column name

FROM tablename

WHERE Condition Operator

(SELECT New Column name

FROM Table name

WHERE Condition);

Q:- "Which employees have a salary greater than Jones salary?"

⇒ SELECT ename, sal

FROM emp

WHERE sal > (SELECT sal

FROM emp

WHERE ename = 'JONES');

// Output

Guidelines for Using Subqueries:-

- ① Enclose subqueries in parentheses.
- ② Place subqueries on the right-side of the comparison operator.
- ③ Using single-row operator with single-row subqueries.
- ④ Using multiple-row operators with multiple-row subqueries.

Types of Subqueries:-

⇒ There are three types.

i. Single-row Subquery

ii. Multiple-row Subquery

iii. Multiple-column Subquery.

Single-row Subqueries:-

⇒ Subqueries that returns single row is called Single row-subquery.

Single-row Comparison Operators:-

<u>Operator</u>	<u>Meaning</u>
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to.

Q:- Display all the employees who are earning more than all the managers.

⇒ SELECT *ename, sal*
FROM *emp*

WHERE *sal > (SELECT MAX(sal)*
FROM *emp*

WHERE job = 'MANAGER');

Q:- Display all the employees who are earning more than any of the managers.

⇒ SELECT *ename, sal*
FROM *emp*

WHERE *sal > (SELECT MIN(sal)*

FROM *emp*

WHERE *job = 'MANAGER');*

// output

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q:- SELECT Employee numbers, job, and salaries of all the Analysts who are earning more than any of the managers.

```
SELECT empno, job, sal
FROM emp
WHERE job = 'ANALYST' AND sal >
      (SELECT min(sal)
       FROM emp
       WHERE job = 'MANAGER');
```

Q:- Select all the employees who work in 'DALLAS'

```
SELECT empno, ename, deptno
FROM emp
WHERE deptno = (SELECT deptno
                FROM emp
                WHERE loc = 'DALLAS');
```

Q:- Display the First maximum salary.

⇒ SELECT MAX(sal)

FROM emp

WHERE sal <=

SELECT MAX(sal)

FROM emp);

Q:- Display the 3rd maximum salary

⇒ SELECT MAX(sal)

FROM emp

WHERE sal <=

(SELECT MAX(sal)

FROM emp

WHERE sal <

(SELECT MAX(sal)

FROM emp

WHERE sal <

(SELECT MAX(sal)

FROM emp)))

// output

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Q. SELECT all the employees who are earning same as 'SMITH'

⇒ SELECT ename, sal

FROM emp

WHERE sal = (SELECT sal

FROM emp

WHERE ename = 'SMITH') AND

ename NOT IN ('SMITH');

Q. Display all the employees who are getting more than the Average salary of all the employees.

⇒ SELECT ename, sal

FROM emp

WHERE sal > (SELECT AVG(sal)

FROM emp);

#Multiple-Row Subquery:-

⇒ Subquery that return multiple rows is called Multiple Row Subquery.

Multiple-Row Comparison Operator:-

Operator

IN

ANY

ALL

Meaning

Equal to any member in the list

Compare value to each value returned by the subquery. It's return true if 'ANY' of the subquery values meet the condition.

Compare value to every value returned by the subquery. It's return true if 'ALL' of the subquery values meet the condition.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

'ANY' Subquery Syntax:-

```
SELECT name_of_column
FROM table_name
WHERE condition operator ANY
(SELECT name_of_column
FROM Table_name
WHERE Condition);
```

'ALL' Subquery Syntax:-

```
SELECT column_name(s)
FROM table_name
WHERE Condition Operator ALL
(SELECT columnname(s)
FROM table_name
WHERE Condition);
```


TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Q: Get all the employees who work in the same departments as of 'SCOTT'

```
=> SELECT ename, deptno
FROM emp
WHERE deptno IN (SELECT deptno
FROM emp
WHERE ename='SCOTT') AND
ename NOT IN('SCOTT');
```

// output

Q: Display all the 'SALESMEN' who are not located at 'DALLAS'.

```
=> SELECT ename, deptno
FROM emp
WHERE job='SALESMEN' AND deptno = ANY
(SELECT deptno
FROM dept
WHERE loc NOT IN('DALLAS'));
```

// output

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q:- SELECT All the 'MANAGER' & 'CLERK' who work in 'ACCOUNT' AND 'MARKETING' Department

```
⇒ SELECT ename, job
FROM emp
WHERE job IN ('MANAGER', 'CLERK') AND
deptno = ANY
(SELECT deptno
FROM dept
WHERE DNAME IN ('ACCOUNT', 'MARKETING'));
```

// output

Q:- Select All the departmental information for all the 'MANAGER'

```
⇒ SELECT
FROM dept
WHERE deptno = ANY
(SELECT deptno
FROM emp
WHERE job = 'MANAGER');
```

// output

Q:- Select department name & location of all the employees working for 'CLARK'.

```
=> SELECT dname, loc  
FROM dept  
WHERE deptno = ANY  
(SELECT deptno  
FROM emp  
WHERE job = 'CLARK');
```

// output

Q:- Select all the employees who work in 'DALLAS'

```
SELECT ename, deptno  
FROM emp  
WHERE deptno = (SELECT deptno  
FROM dept  
WHERE loc = 'DALLAS');
```

// output

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Q:- SELECT the employee name who get the department wise lowest salary

```
=> SELECT ename, sal
FROM emp
WHERE sal = ANY
      (SELECT MIN(sal)
       FROM emp
       GROUP BY deptno);
```

// output

Q:- Find the employee name, job, no who getting more than the Average salary of department wise

```
=> SELECT ename, job, empno
FROM emp
WHERE sal > ALL (SELECT AVG(sal)
                  FROM emp
                  GROUP BY deptno);
```

// output

Joining of Tables

Q:- What is join?

⇒ It's used to combine rows from two or more tables, based on a related column between them.

Syntax of Query:-

SELECT t1.column, t2.column

FROM table1 t1, table2 t2

WHERE t1.condition = t2.condition

Cartesian-product:-

⇒ The cartesian product is really a cross-join which return all the rows in all the tables listed in a query. Each row in the 1st table is paired with all the rows in the 2nd table.

* Cartesian product is formed:

1. A join condition is omitted

2. A join condition is invalid.

Note:- To avoid it, always include a valid join condition in a WHERE clause.

Types of joins:-

1. Equi-join

2. Non-equi-join

3. Outer-join

4. Self-join

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Equi-Join:-

⇒ It's also known as inner-join. The Equi-Join creates a new result table when it combines the values of columns of two-tables on the basis of join-predicate.

1st Query Syntax:-

```
SELECT Column_List  
FROM table1, table2, ...  
WHERE table1.Column_Name =  
table2.Column_Name;
```

OR 2nd Query Syntax:-

```
SELECT Column_List  
FROM table1  
JOIN table2  
ON table1.Column_Name = table2.Column_Name;
```

Non-Equi-Join:

⇒ A join that matches column values from different tables based on inequality. It's performs a join using comparison operator like $>$, $<$, $>=$, $<=$, with Condition.

Query Syntax:-

SELECT column list

FROM table1, table2

WHERE table1.columnName [$>$ | $<$ | $>=$ | $<=$]
table2.columnName

Ex:-

⇒ SELECT e.ename, e.sal, e.deptno, s.grade
FROM emp e, salgrade s
WHERE e.sal
BETWEEN s.lo_sal AND s.hi_sal;

#Outer Joins:-

⇒ It's used to return all the records that have values in either the left or right table or both tables.

There are few types of outer joins:-

1. Left join

2. Right join

3. Full Outer join

LEFT JOIN:-

⇒ It's return only unmatched rows from left table, as well as matched rows from both tables.

Query Syntax:-

SELECT columns list

FROM table 1 +1, table 2 +2

WHERE +1.columnname(+) = +2.column name

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Ex:-

```
=> SELECT e.ename, d.deptno, d.dname  
FROM emp e, dept d  
WHERE e.deptno(+) = d.deptno  
ORDER BY d.deptno;
```

// output

RIGHT JOIN:-

=> It returns only unmatched rows from right table, as well as matched rows in both tables.

Query Syntax:-

```
SELECT Column-Names  
FROM table1 +1, table2 +2  
WHERE +1.columnname = +2.columnname(+);
```

FULL OUTER JOIN:-

- ⇒ Returns Unmatched rows from both tables, as well as return matched rows in both tables.

Query Syntax:-

SELECT Column-List

FROM table 1

FULL OUTER JOIN table 2

ON table 1.column = table 2.column;

Self-Join:-

- ⇒ In Self-join a table is joined with itself. Each row of a table is combined with itself and every other row of the table.

Query Syntax:-

SELECT a.column1, b.column2

FROM table 1 a, table 1 b

WHERE a.column = b.column

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Exercise of joining query

Q:- Select All the employee who work in 'DALLAS'

```
⇒ SELECT e.ename, d.loc  
FROM emp e, dept d  
WHERE d.loc = 'DALLAS'  
AND d.deptno = e.deptno ;
```

OR

```
⇒ SELECT e.ename, d.loc  
FROM emp e  
JOIN dept d  
ON d.deptno = e.deptno  
WHERE d.loc = 'DALLAS';
```

// output

Q:- Select the records from emp table that have not matching records in dept table.

⇒ SELECT *

FROM emp e, dept d

WHERE e.deptno (+) = d.deptno ;

// output

Q:- Display all departmental information for all the existing employees and if a department has no employees display it as "No Employees".

⇒ SELECT d.deptno, d.dname, d.loc,
NVL(e.ename, 'No Employees')

FROM emp e, dept d

WHERE e.deptno (+) = d.deptno ;

// output

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q:- Get all the matching & non-matching records from both tables.

⇒ SELECT *
FROM emp e

FULL OUTER JOIN dept d

ON e.deptno = d.deptno;

// output

Q:- Get only non-matching records from dept table (matching records shouldn't be selected).

⇒ SELECT *

FROM emp e

FULL OUTER JOIN dept d

ON e.deptno = d.dept

WHERE e.deptno IS NULL;

// output

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Q. Select all the employees from emp tables with their manager names, and if an employee does not have a manager, display him as 'CEO'

```
⇒ SELECT e.ename, NVL(m.ename, 'CEO')
```

```
FROM emp e, emp m
```

```
WHERE m.empno(+) = e.mgr
```

// output

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q:- Get all the employees who work in the same departments as of 'SCOTT'

```
⇒ SELECT e.ename, d.deptno
FROM emp e, emp d
WHERE e.deptno = d.deptno
AND e.ename = 'SCOTT';
```

// output

Q:- Display all the employees who have joined before their manager.

```
⇒ SELECT e.ename
FROM emp e, emp m
WHERE m.HIREDATE < e.HIREDATE
AND m.MGR = e.EMPNO
```

// output

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Q:- List all the employees who are earning more than their manager.

```
⇒ SELECT e.ename  
FROM emp e, emp m  
WHERE e.sal > m.sal  
AND m.empno = e.mgr;
```

// Output

Q:- Fetch all the employees who earning same salaries.

```
⇒ SELECT e.ename, e.sal, d.sal, d.ename  
FROM emp e, emp d  
WHERE e.sal = d.sal  
AND e.empno <> d.empno;
```

// Output

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q:- SELECT all the dept information for all the managers

⇒ SELECT e.mgr, d.*

FROM emp e, dept d

WHERE e.deptno = d.deptno

AND e.mgr IS NOT NULL;

// output

Q:- Select dept all information of all the employees working for 'CLERK'

⇒ SELECT d.*

FROM emp e, dept d

WHERE d.deptno = e.deptno

AND e.job = 'CLERK';

// output

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Views:-

⇒ A subset of a database and is based on a query that runs on one or more database tables.

Why use Views:-

- ⇒ (i) To restrict database access.
- (ii) To make complex query easy
- (iii) To allow data independence.
- (iv) To present different views of the same data.

Simple Views & Complex Views

<u>Feature</u>	<u>Simple Views</u>	<u>Complex Views</u>
1. No. of tables	one	one or many
2. Contain function	No	yes
3. Contain group of data	No	yes
4. DML through view	yes	Not always

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

How to Create a View :-

Query Syntax:-

```
CREATE VIEW viewname  
AS SELECT column1, column2, ....  
FROM tablename  
WHERE condition;
```

Q:- Create a view, EMPVU10, that contains details of employee in department 10.

```
⇒ CREATE VIEW EMPVU10  
AS SELECT empno, ename, job  
FROM emp  
WHERE deptno = 10;
```

Q:- Describe the structure of the view by using the command.

```
⇒ DESC empvu10;
```

//output

Create a View by
using column aliases:-
~~~~~

```
=> CREATE VIEW salvu30  
AS SELECT empno EMPLOYEE_NUMBER,  
          ename NAME, sal SALARY  
FROM emp  
WHERE deptno = 30;
```

Q:- Display all the information from salvu30

```
=> SELECT *  
FROM salvu30;
```

//Output



TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_ DATE : / /

## # Modifying a View :-

### Query Syntax :-

CREATE OR REPLACE VIEW viewname.

AS SELECT column1 alias, column2 alias2

FROM table-name

WHERE condition;

Q:- modify the EMPVU10, add an alias for each column name.

⇒ CREATE OR REPLACE VIEW Empvu10

AS SELECT empno employee number, ename employee name,

job job title

FROM emp

WHERE deptno = 10;

// output :-

### #Creating a Complex View:-

⇒ Create a Complex view that contains group function to display values from two tables.

⇒ CREATE VIEW deptsumv

AS SELECT d.dname NAME, MIN(e.sal) MINSAL,  
MAX(e.sal) MAXSAL, AVG(e.sal) AVGSAL

FROM emp e, dept d

WHERE e.deptno = d.deptno

GROUP BY d.dname;

### #RULES for Performing DML operation on a view:-

⇒ ① We can perform DML operation on Simple Views.

② We can't remove a row if view contains the following

① Group function.

② A group by clause

③ The DISTINCT keyword.

TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_ DATE : / /

# Create VIEW Using the 'WITH  
CHECK OPTION' clause:-

Query Syntax:-

```
CREATE VIEW viewname  
AS SELECT column1, column2, ...  
FROM table name  
WHERE Condition  
WITH CHECK OPTION CONSTRAINT viewname CK;
```

Ex:-

```
CREATE VIEW empvu20  
AS SELECT *  
FROM emp  
WHERE deptno = 20  
WITH CHECK OPTION CONSTRAINT empvu20 CK;
```



TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_

DATE : / /

### #READ ONLY:-

⇒ We can ensure that no DML operation occur by adding 'WITH READ ONLY' option in our view definition.

### Query Syntax

```
CREATE VIEW view-name  
AS SELECT table1, table2, ...  
FROM table-name  
WHERE condition  
WITH READ ONLY;
```

### Ex:-

```
CREATE VIEW empview10  
AS SELECT *  
FROM emp  
WHERE deptno = 10;  
WITH READ ONLY;
```

TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_ DATE : / /

## # Removing a View:-

### Query Syntax:-

```
DROP VIEW VIEWNAME;
```

## # Practice Query:-

Q:- Create a view called 'EMP\_VU' based on the empno, ename, deptno from the 'Emp' table, Change the heading for the "employee" name to 'EMPLOYEE'

```
=> CREATE VIEW EMP_VU  
AS SELECT empno, ename EMPLOYEE, deptno  
FROM emp;
```

// output

Q:- Display the contents of the 'EMP\_VU' view.

```
=> SELECT *  
FROM EMP_VU;
```

// output



Q:- using the view EMP\_VU, enter a query to display all the employee names and deptno

```
⇒ SELECT EMPLOYEE, deptno  
FROM EMP_VU;
```

//output

Q:- Create a view named DEPT20 that contains the empno, ename, deptno for all employees in dept 20. Label the view column EMPLOYEE-ID, EMPLOYEE, DEPARTMENT-ID, Do not allow an employee to be reassigned to another department through the view.

```
⇒ CREATE VIEW DEPT20  
AS SELECT empno EMPLOYEE-ID,  
ename EMPLOYEE,  
deptno DEPARTMENT-ID
```

```
FROM emp
```

```
WHERE deptno = 20
```

```
WITH CHECK CONSTRAINT DEPT20CK;
```



TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_ DATE : / /

Q:- Create a view called SALARY-VU based on the employee name, department name, salary, and salary grade for all employees. Label the columns Employee, Department, Salary, Grade, respectively.

⇒ CREATE VIEW SALARYVU

AS SELECT e.ename EMPLOYEE,

d.dname DEPARTMENT,

e.sal SALARY,

s.grade Grade

FROM emp e, dept d, salgrade s

WHERE e.deptno = d.deptno

AND e.sal BETWEEN s.losal AND s.hisal;

## # SEQUENCE:-

⇒ A database object that generates and produce integer values in sequential order.

### Query Syntax:-

CREATE SEQUENCE *sequence name*

START WITH *n*

INCREMENT BY *n*

MAXVALUE *n*

MINVALUE *n*

CYCLE

CACHE *n*;

OR

CREATE SEQUENCE *Sequence name*

START WITH *n*

INCREMENT BY *n*

NOMAXVALUE

NOMINVALUE

NOCYCLE;

NOCACHE;

TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_ DATE : / /

Q:- Create a sequence named DEPT-DEPTNO to be used for the primary key of the DEPT table. DO not use cycle option.

```
CREATE SEQUENCE dept-deptno  
INCREMENT BY 10  
START WITH 50  
NOMAXVALUE  
NOMINVALUE  
NOCYCLE  
NOCACHE;
```

#Confirming Sequences:-

Query Syntax:-

```
SELECT sequence_name, min_value, max_value,  
       increment_by, last_number  
FROM user-sequences;
```



TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_

DATE : / /

### #NEXTVAL & CURRVAL:-

⇒ **NEXTVAL** returns the next available sequence value.

⇒ **CURRVAL** returns the current sequence value.

### #Using a Sequence

Ex:-

⇒ Insert into **dept**

VALUES (**dept-deptno.NEXTVAL**, 'MARKETING', 'San-Diego')

Q:- view the **CURRVAL** for the **DEPT-DEPTNO**

⇒ SELECT **dept-deptno.CURRVAL**

FROM **dual;**

// output

TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_

DATE : / /

## # Modifying a sequence:-

### Query Syntax:-

```
ALTER SEQUENCE sequence name  
INCREMENT BY n  
MAXVALUE n  
NO MINVALUE  
NO CACHE  
NO CYCLE;
```

## # Removing a Sequence:-

### Query Syntax:-

```
DROP SEQUENCE sequence name;
```

### # practice Sequence Queries-

Q:- Create a sequence to used with primary key column of the department table. The sequence should start at 60 and have a maximum value 200. Have your sequence increment by 10 number. NAME the sequence DEPT-ID-SEQ.

```
=> CREATE SEQUENCE DEPTIDSEQ  
    INCREMENT BY 10  
    MAXVALUE 200  
    START WITH 60
```

Q:- Write a script to display the following information about your sequences. Sequence-name, max-value, min-value increment-by and last-number.

```
=> SELECT sequence_name, maxvalue, min_value,  
        increment_by, last_number  
FROM user_sequences;
```



TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_ DATE : / /

## # Relational - Algebra :-

⇒ A procedural query language, which takes instances of relation as input and yield instance of relations as output.

## # Six - Basic operators :-

1. select :  $\sigma$

2. project :  $\Pi$

3. Union :  $\cup$

4. Set difference :  $-$

5. Cartesian product :  $\times$

6. rename :  $\rho$

→ The operators take one or two relations as input and produce a new relation as a result.

## # SELECT - Operation ( $\sigma$ )

Notation:-

$\sigma_p(r)$

Where

$\sigma$  = for selection

$r$  = for relation

$p$  = propositional logic formula.

Q:- SELECT tuples from books where subject in 'database'

$\Rightarrow \sigma_{\text{subject} = \text{"database"}}(\text{Books})$

Q:- SELECT tuples from books where subjects is 'database' and price is 450 or those books published after 2010

$\Rightarrow \sigma_{\text{subject} = \text{"Database"} \text{ and price} = \text{"450"} \text{ or year} > \text{"2010"}}(\text{Books})$

TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_

DATE : / /

## # project operation ( $\pi$ )

Notation:-

$$\pi A_1, A_2, A_n (r)$$

where,

$A_1, A_2 =$  Attribute names

$r =$  relation.

Note:-

Duplicate rows are automatically eliminated as relation in a set.

Q:- Find the Customer who live in Harrison

$$\Rightarrow \pi_{\text{customer-name}} (\sigma_{\text{customer-city} = \text{"Harrison"}} (\text{Customer}))$$

Q:- SELECT & projects columns name as Subject and Author from the relation Book

$$\Rightarrow \pi_{\text{subject, author}} (\text{Book})$$



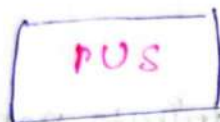
TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_ DATE : / /

## ## Union - Operation ( $\cup$ )

Notation:-



Where:-

$R$  and  $S$  are either database relation or relation result set.

Condition:-

1.  $R$  and  $S$  must have the same number of attributes.
2. Attributes domain must be compatible.
3. Duplicate tuple are automatically eliminated.

Q:- projects the names of the Authors who have have either written a book or an article or both.

$\Rightarrow$   $\text{Authors(Books)} \cup \text{Authors(Article)}$

## #Set - Difference(-)

⇒ The result of set difference operation is tuples, which are present in one relation but are not in second relation.

Notation:-

$$R - S$$

Where,

Finds all tuples that are present in  $R$  but not in  $S$ .

Q:- provides the name of Authors who have written book but not articles.

$$\Rightarrow \pi_{\text{author}}(\text{Book}) - \pi_{\text{author}}(\text{Articles})$$

## # Cartesian Product ( $\times$ )

$\Rightarrow$  Combines information of two different relations into one.

Notation:-

$$R \times S$$

Q:- Yields a relation, which show all the books and articles written by tutorialspoint

$\Rightarrow$   $\text{Author} = \text{"tutorialspoint"} (\text{Book} \times \text{Articles})$

## # Rename Operation ( $\rho$ )

Notation:-

$$\rho_x(E)$$

where the result of expression  $E$  is saved with name of  $x$ .



TOPIC NAME : \_\_\_\_\_

DAY : \_\_\_\_\_

TIME : \_\_\_\_\_ DATE : / /

## # Joining Operation:-

Ex:-

$\sigma(R \times S)$

and

$\sigma_{A=B}(R \times S)$

Q:- Find the names of all customers who have a loan at the perryridge branch.

$\Rightarrow \pi_{\text{customer.name}}(\sigma_{\text{branch.name} = \text{"Perryridge"}}(\sigma_{\text{borrower.name} = \text{loan.name}}(\text{borrower} \times \text{loan})))$

## # Natural join Operation:-

Ex:-

$\sigma(R \bowtie S)$

# Normalization

⇒ Normalization is the process of organizing data in a database.

## What is Normalization?

- Normalization is the process of organizing the data in the database.
- Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
- Normalization divides the larger table into smaller and links them using relationships.
- The normal form is used to reduce redundancy from the database table.

**Data modification anomalies can be categorized into three types:**

- **Insertion Anomaly:** Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.
- **Deletion Anomaly:** The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.
- **Updatation Anomaly:** The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

## Types of Normal Forms:

Normalization works through a series of stages called Normal forms. The normal forms apply to individual relations. The relation is said to be in particular normal form if it satisfies constraints.

Following are the various types of Normal forms:

|                           | 1NF                        | 2NF                                     | 3NF                                                   | 4NF                                                                      | 5NF                                                                                         |
|---------------------------|----------------------------|-----------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Decomposition of Relation | R                          | R <sub>11</sub><br>R <sub>12</sub>      | R <sub>21</sub><br>R <sub>22</sub><br>R <sub>23</sub> | R <sub>31</sub><br>R <sub>32</sub><br>R <sub>33</sub><br>R <sub>34</sub> | R <sub>41</sub><br>R <sub>42</sub><br>R <sub>43</sub><br>R <sub>44</sub><br>R <sub>45</sub> |
| Conditions                | Eliminate Repeating Groups | Eliminate Partial Functional Dependency | Eliminate Transitive Dependency                       | Eliminate Multi-values Dependency                                        | Eliminate Join Dependency                                                                   |

| Normal Form         | Description                                                                                                             |
|---------------------|-------------------------------------------------------------------------------------------------------------------------|
| <a href="#">1NF</a> | A relation is in 1NF if it contains an atomic value.                                                                    |
| <a href="#">2NF</a> | A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key. |
| <a href="#">3NF</a> | A relation will be in 3NF if it is in 2NF and no transition dependency exists.                                          |
| BCNF                | A stronger definition of 3NF is known as Boyce Codd's normal form.                                                      |
| <a href="#">4NF</a> | A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.                      |
| <a href="#">5NF</a> | A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.             |



## First Normal Form (1NF)

- A relation will be 1NF if it contains an atomic value.
- It states that an attribute of a table cannot hold multiple values. It must hold only single-valued attribute.
- First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

**Example:** Relation EMPLOYEE is not in 1NF because of multi-valued attribute EMP\_PHONE.

- **EMPLOYEE table:**

| EMP_ID | EMP_NAME | EMP_PHONE                 | EMP_STATE |
|--------|----------|---------------------------|-----------|
| 14     | John     | 7272826385,<br>9064738238 | UP        |
| 20     | Harry    | 8574783832                | Bihar     |
| 12     | Sam      | 7390372389,<br>8589830302 | Punjab    |

- The decomposition of the EMPLOYEE table into 1NF has been shown below:

| EMP_ID | EMP_NAME | EMP_PHONE  | EMP_STATE |
|--------|----------|------------|-----------|
| 14     | John     | 7272826385 | UP        |
| 14     | John     | 9064738238 | UP        |
| 20     | Harry    | 8574783832 | Bihar     |
| 12     | Sam      | 7390372389 | Punjab    |
| 12     | Sam      | 8589830302 | Punjab    |

## Second Normal Form (2NF)

- In the 2NF, relational must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on the primary key

**Example:** Let's assume, a school can store the data of teachers and the subjects they teach. In a school, a teacher can teach more than one subject.

**TEACHER table**

| TEACHER_ID | SUBJECT   | TEACHER_AGE |
|------------|-----------|-------------|
| 25         | Chemistry | 30          |
| 25         | Biology   | 30          |
| 47         | English   | 35          |
| 83         | Math      | 38          |
| 83         | Computer  | 38          |

In the given table, non-prime attribute TEACHER\_AGE is dependent on TEACHER\_ID which is a proper subset of a candidate key. That's why it violates the rule for 2NF.

To convert the given table into 2NF, we decompose it into two tables:

**TEACHER\_DETAIL table:**

| TEACHER_ID | TEACHER_AGE |
|------------|-------------|
| 25         | 30          |
| 47         | 35          |
| 83         | 38          |

**TEACHER\_SUBJECT table:**

| TEACHER_ID | SUBJECT   |
|------------|-----------|
| 25         | Chemistry |
| 25         | Biology   |
| 47         | English   |
| 83         | Math      |
| 83         | Computer  |

## Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

A relation is in third normal form if it holds at least one of the following conditions for every non-trivial function dependency  $X \rightarrow Y$ .

1. X is a super key.
2. Y is a prime attribute, i.e., each element of Y is part of some candidate key.

**Example:**



**EMPLOYEE\_DETAIL table:**

| EMP_ID | EMP_NAME  | EMP_ZIP | EMP_STATE | EMP_CITY |
|--------|-----------|---------|-----------|----------|
| 222    | Harry     | 201010  | UP        | Noida    |
| 333    | Stephan   | 02228   | US        | Boston   |
| 444    | Lan       | 60007   | US        | Chicago  |
| 555    | Katharine | 06389   | UK        | Norwich  |
| 666    | John      | 462007  | MP        | Bhopal   |

**Super key in the table above:**

1.{EMP\_ID}, {EMP\_ID, EMP\_NAME}, {EMP\_ID, EMP\_NAME, EMP\_ZIP} ....so on

**Candidate key:** {EMP\_ID}

**Non-prime attributes:** In the given table, all attributes except EMP\_ID are non-prime.

Here, EMP\_STATE & EMP\_CITY dependent on EMP\_ZIP and EMP\_ZIP dependent on EMP\_ID. The non-prime attributes (EMP\_STATE, EMP\_CITY) transitively dependent on super key (EMP\_ID). It violates the rule of third normal form.

That's why we need to move the EMP\_CITY and EMP\_STATE to the new <EMPLOYEE\_ZIP> table, with EMP\_ZIP as a Primary key.

**EMPLOYEE table:**

| EMP_ID | EMP_NAME  | EMP_ZIP |
|--------|-----------|---------|
| 222    | Harry     | 201010  |
| 333    | Stephan   | 02228   |
| 444    | Lan       | 60007   |
| 555    | Katharine | 06389   |
| 666    | John      | 462007  |

**EMPLOYEE\_ZIP table:**

| EMP_ZIP | EMP_STATE | EMP_CITY |
|---------|-----------|----------|
| 201010  | UP        | Noida    |
| 02228   | US        | Boston   |
| 60007   | US        | Chicago  |
| 06389   | UK        | Norwich  |
| 462007  | MP        | Bhopal   |

## Boyce Codd normal form (BCNF)

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency  $X \rightarrow Y$ , X is the super key of the table.
- For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

**Example:** Let's assume there is a company where employees work in more than one department.

**EMPLOYEE table:**

| EMP_ID | EMP_COUNTRY | EMP_DEPT   | DEPT_TYPE | EMP_DEPT_NO |
|--------|-------------|------------|-----------|-------------|
| 264    | India       | Designing  | D394      | 283         |
| 264    | India       | Testing    | D394      | 300         |
| 364    | UK          | Stores     | D283      | 232         |
| 364    | UK          | Developing | D283      | 549         |

**In the above table Functional dependencies are as follows:**

1.  $EMP\_ID \rightarrow EMP\_COUNTRY$
2.  $EMP\_DEPT \rightarrow \{DEPT\_TYPE, EMP\_DEPT\_NO\}$

**Candidate key: {EMP-ID, EMP-DEPT}**

The table is not in BCNF because neither EMP\_DEPT nor EMP\_ID alone are keys.

To convert the given table into BCNF, we decompose it into three tables:

**EMP\_COUNTRY table:**

| EMP_ID | EMP_COUNTRY |
|--------|-------------|
| 264    | India       |
| 264    | India       |

**EMP\_DEPT table:**

| EMP_DEPT   | DEPT_TYPE | EMP_DEPT_NO |
|------------|-----------|-------------|
| Designing  | D394      | 283         |
| Testing    | D394      | 300         |
| Stores     | D283      | 232         |
| Developing | D283      | 549         |

**EMP\_DEPT\_MAPPING table:**

| EMP_ID | EMP_DEPT |
|--------|----------|
| D394   | 283      |
| D394   | 300      |
| D283   | 232      |
| D283   | 549      |



### Functional dependencies:

1.  $EMP\_ID \rightarrow EMP\_COUNTRY$
2.  $EMP\_DEPT \rightarrow \{DEPT\_TYPE, EMP\_DEPT\_NO\}$

### Candidate keys:

For the first table:  $EMP\_ID$   
For the second table:  $EMP\_DEPT$   
For the third table:  $\{EMP\_ID, EMP\_DEPT\}$

Now, this is in BCNF because left side part of both the functional dependencies is a key.

## Fourth normal form (4NF)

- A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
- For a dependency  $A \twoheadrightarrow B$ , if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

### Example

#### STUDENT

| STU_ID | COURSE    | HOBBY   |
|--------|-----------|---------|
| 21     | Computer  | Dancing |
| 21     | Math      | Singing |
| 34     | Chemistry | Dancing |
| 74     | Biology   | Cricket |
| 59     | Physics   | Hockey  |

The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity. Hence, there is no relationship between COURSE and HOBBY.

In the STUDENT relation, a student with STU\_ID, **21** contains two courses, **Computer** and **Math** and two hobbies, **Dancing** and **Singing**. So there is a Multi-valued dependency on STU\_ID, which leads to unnecessary repetition of data.

So to make the above table into 4NF, we can decompose it into two tables:

#### STUDENT\_COURSE

| STU_ID | COURSE    |
|--------|-----------|
| 21     | Computer  |
| 21     | Math      |
| 34     | Chemistry |
| 74     | Biology   |
| 59     | Physics   |

#### STUDENT\_HOBBY

| STU_ID | HOBBY   |
|--------|---------|
| 21     | Dancing |
| 21     | Singing |
| 34     | Dancing |
| 74     | Cricket |
| 59     | Hockey  |

## Fifth normal form (5NF)

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- 5NF is also known as Project-join normal form (PJ/NF).

### Example

| SUBJECT   | LECTURER | SEMESTER   |
|-----------|----------|------------|
| Computer  | Anshika  | Semester 1 |
| Computer  | John     | Semester 1 |
| Math      | John     | Semester 1 |
| Math      | Akash    | Semester 2 |
| Chemistry | Praveen  | Semester 1 |

In the above table, John takes both Computer and Math class for Semester 1 but he doesn't take Math class for Semester 2. In this case, combination of all these fields required to identify a valid data.

Suppose we add a new Semester as Semester 3 but do not know about the subject and who will be taking that subject so we leave Lecturer and Subject as NULL. But all three columns together acts as a primary key, so we can't leave other two columns blank.

So to make the above table into 5NF, we can decompose it into three relations P1, P2 & P3:



**P1**

| SEMESTER   | SUBJECT   |
|------------|-----------|
| Semester 1 | Computer  |
| Semester 1 | Math      |
| Semester 1 | Chemistry |
| Semester 2 | Math      |

**P2**

| SUBJECT   | LECTURER |
|-----------|----------|
| Computer  | Anshika  |
| Computer  | John     |
| Math      | John     |
| Math      | Akash    |
| Chemistry | Praveen  |

**P3**

| SEMSTER    | LECTURER |
|------------|----------|
| Semester 1 | Anshika  |
| Semester 1 | John     |
| Semester 1 | John     |
| Semester 2 | Akash    |
| Semester 1 | Praveen  |